AI & Software Engineering Workshop

Week 1, Day 3: Commands and Memory — Windows

Edik Simonian, Summer 2026

Decorators in 60 seconds

```
@bot.message_handler(commands=["start"], func=is_allowed)
def cmd_start(message):
    ...
```

- A decorator **registers** your function with the bot
- *"When a message matches `/start`, call this function"*
- You don't call `cmd_start` yourself; the bot does
- That's all you need to know to add any command

The built-ins: read before you write

Command	What it does
`/start`	Welcome message (you already edited it)
`/help`	Lists commands (you edited it yesterday)
`/reset`	Clears your conversation history
`/about`	Model, storage, and hosting info

`/reset` clears *history*. Wait, what history? **That's the second half of today.**

Live-code: `/joke`

A hardcoded joke tells the same one forever. Let's make the command **think**:

```
@bot.message_handler(commands=["joke"], func=is_allowed)
def cmd_joke(message):
    reply = ask_ai(message.from_user.id, "Tell one short, clean programming joke.")
    bot.send_message(message.chat.id, reply)
```

`ask_ai(user_id, prompt)` sends a one-off prompt and returns the reply — **a fresh joke every time**, no list to maintain.

1. Add the handler to `bot/handlers.py`
2. Add `/joke` to `/help`
3. Restart, test

Your turn: pick one

Each asks the AI — fresh output every time, just like `/joke`:

- `/quote`: an original motivational line
- `/fact`: a surprising fact (let the AI surprise you)
- `/compliment`: brighten someone's day
- `/roll`: a dice roll — the **odd one out**, pure Python (`random.randint`), no AI

Same recipe: handler → `/help` entry → restart → test.

Build the first by hand; pair with Claude Code on the next, but you explain it in the review circle.

Level up: pass your words to the AI

`/roast <name>`: read the words after the command, send them to the AI:

```
@bot.message_handler(commands=["roast"], func=is_allowed)
def cmd_roast(message):
    name = message.text.split(maxsplit=1)[1] if " " in message.text else "you"
    reply = ask_ai(message.from_user.id, f"Write a short, playful, friendly roast of {name}.")
    bot.send_message(message.chat.id, reply)
```

`message.text.split()`: that's argument parsing.

(The repo's `/model` handler does the same trick; it activates in Week 2.)

Lock it in with a test

- Open `tests/test_handlers.py`: see how `/start` is tested
- Write the same shape of test for **your** command
- `.\make.ps1 test`: green locally
- `git push`: green on GitHub Actions too

A command without a test is a command that breaks silently later.

Code review circle

1. Rotate one seat to your left, to your neighbor's screen
2. Read the handler on that screen **aloud**, line by line
3. Explain what each line does; ask if you can't
4. One compliment, one suggestion, then rotate back

Reading other people's code is half of real software engineering.

The problem

1. Tell your bot: "*my favorite color is blue*"
2. Then ask, in your **very next message**: "*what's my favorite color?*"
3. **It has no idea.**

Each reply forgets the one before it, that's the "stateless mode" notice from Day 1.
Commands done; **now we give the bot a memory.**

Key-value store = lockers

- A wall of lockers 🔐
- **Key** = the locker number → `chat:12345`
- **Value** = whatever you put inside → the conversation
- `get(key)` / `set(key, value)`: that's the whole API
- Ours also has **TTL**: lockers that empty themselves after 30 days

Redis, DynamoDB, memcached, same idea. Ours is SQLite: a database in a single file.

Turn it on

`.env`:

```
SQLITE_PATH=./bot.db
```

- Restart → tell it your favorite color → restart again → ask
- **It remembers.** The file `bot.db` appeared: that's the memory
- No signup, no server, no credit card. It's just a file
- Delete the line → graceful fallback to stateless mode (try it)

How the bot remembers conversations

The model is **stateless**: each API call forgets the last. The memory is *our* code, not the model.

```
# bot/ai.py — runs on every message
history = get_history(user_id)          # load past turns from bot.db
history.append({"role": "user", "content": text})

messages = [{"role": "system", "content": SYSTEM_PROMPT}, *history]
reply = generate(user_id, messages)     # replay the whole chat

history.append({"role": "assistant", "content": reply})
save_history(user_id, history)          # write back: last 20, 30-day TTL
```

Keyed by `chat:<your id>`, the list lives in `bot.db`, so it **survives restarts**.

Live-code: `/remember` and `/recall`

```
@bot.message_handler(commands=["remember"], func=is_allowed)
def cmd_remember(message):
    note = message.text.split(maxsplit=1)[1] if " " in message.text else ""
    store.set(f"note:{message.from_user.id}", note)
    bot.send_message(message.chat.id, "Saved!")
```

`/recall` is the mirror image: `store.get(f"note:{...}")`.

Solo build: a full notes feature

- `/remember <text>`: add a note (not replace!)
- `/recall`: list **all** saved notes
- `/forget`: clear them

Hint: the store saves **strings**. To keep a list, `json.dumps` it on the way in, `json.loads` it on the way out.

Today → Tomorrow

Today your bot has commands that **think**, and a memory that **survives restarts** — and you found out what `/reset` actually resets.

Tomorrow: deploy. The bot goes live on the internet and updates on every `git push`, so it stops needing your laptop.